RTL-Universe: A Project-Scale Benchmark for LLM Coding Agents on Hardware Description Languages

David Andrews
dha@xoba.com

Saketh Reddy
yssaketh@gmail.com

Abstract

Existing LLM benchmarks for hardware description languages (HDL) test *single-module* Verilog generation from natural-language specifications and are now near-saturated: VerilogEval, RTLLM, and CVDP all sit above 90% pass rate for current models. We argue this is the wrong measurement: real hardware engineering happens at the project scale, where dozens of files cooperate through bus protocols, build systems, and CI test suites. We introduce RTL-UNIVERSE, a benchmark of eleven open-source HDL projects (Ibex, VeeR EL2, NVDLA, Coral NPU, OpenPiton, PULP common cells, Secworks AES) in which the implementation files of one design unit are stripped and an LLM coding agent is asked to restore them so that the project's unmodified upstream CI suite passes. We evaluate seven models across three coding-agent harnesses (Claude Code, Codex CLI, OpenCode) on a 7×11 matrix. The strongest agent passes 6/11 tasks; four of the seven pass 0–2/11. A forensic audit of agent transcripts further shows that 10 of the 15 perfect scores in the run are achieved by exploiting either an unblocked sandbox egress path or a verifier flaw rather than by writing the design — on the eight tasks whose verifiers we trust, the strongest agent passes 2/11 tasks. We report the benchmark, the harness, the results, the bypass taxonomy, and concrete recommendations for future hardware agent benchmarks.

1 Introduction

LLM coding agents have improved fast on general-purpose software-engineering benchmarks. SWE-bench [Jimenez et al., 2024] reported $\sim$2% pass rate at launch; current top agents pass $\sim$70%. Hardware description languages have followed a similar curve at the *module* level: VerilogEval [Liu et al., 2023], RTLLM [Lu et al., 2024], and the recent CVDP suite [NVIDIA Research, 2024] all report 90%+ pass rates for current frontier models. Where SWE-bench updated to "Verified" [OpenAI, 2024b] and then to ProgramBench [OpenAI, 2024a] as agents saturated the simpler regime, no equivalent harder benchmark exists for HDL. We propose one.

The premise of RTL-UNIVERSE is that real hardware engineering takes place at the *project* scale: an engineer joining an open-source RISC-V core or DL accelerator works inside a directory of dozens of source files, a handful of testbenches, a build system that calls Verilator or Bazel or FuseSoC, and a CI script. Their job is rarely "write a UART transmitter from a paragraph" — it is to make a specific change, or fix a bug, or re-implement a missing module against the rest of the project. We capture this by taking real open-source HDL projects, deleting one design unit's implementation files, and asking an agent to restore them so the project's own CI passes.

Our contributions are:

[nosep]

1. **A benchmark** of 11 restoration tasks built on real open-source projects spanning two RISC-V cores (Ibex, VeeR EL2), a deep-learning accelerator (NVDLA), a small NPU (Coral NPU), peripherals (OpenPiton ESL, AES), and a cell library (PULP common cells)

(Section 3). Together they require restoring 4–280 source files per task and exercise build systems including Verilator, Icarus, FuseSoC, Bazel, and cocotb.

2. **A multi-agent harness** that runs Claude Code, Codex CLI, and OpenCode under a DNS sinkhole, with per-account concurrency caps for subscription-billed agents and a disk-/process-supervision layer that we found necessary in practice (Section 4).

3. **An empirical evaluation** of seven models on the 7×11 matrix (Section 5). The strongest agent (GPT-5.5, reasoning effort `xhigh`) reports 6/11 perfect-score passes. The OpenCode-on-OpenRouter open-weight models (Kimi K2.6, DeepSeek V4-Pro, GLM-5.1, Qwen 3.6-27B) cluster at 0–2 perfects. Several tasks (`coralnpu-full`, `nvdla-e2e`, `openpiton-e2e`, `pulp-common-cells-full`) are passed by zero or one model.

4. **An audit** of the agent transcripts that catalogues eight ways the agent *can* score without doing the task — six varieties of source-code download bypass, plus two verifier exploits we discovered in the upstream test suites (Section 6). We treat this as a quality-control finding rather than a moral one: the same audit gives a re-scored leaderboard on the eight tasks whose verifiers we trust.

RTL-UNIVERSE is the latest in a sequence of internal benchmarks the authors have built (a more chronological account appears in our companion logbook); *this paper focuses on the benchmark itself* and what running it tells us about current agents on project-scale hardware tasks.

2 Related Work

Module-level HDL benchmarks. VerilogEval [Liu et al., 2023] (156 problems), RTLLM [Lu et al., 2024] (50 problems), and VeriGen [Thakur et al., 2023] grade single-module Verilog generation from natural-language descriptions. Test suites are 1–10 lines and the design is self-contained. CVDP [NVIDIA Research, 2024] (783 problems) extends this with non-agentic and agentic variants but stays at the module scale. None of these benchmarks asks the agent to operate on an existing build system or write code that is then *linked into a larger project*.

Agentic SWE benchmarks. SWE-bench [Jimenez et al., 2024] runs agents against real Python GitHub issues; SWE- agent [Yang et al., 2024] introduced the agent-computer interface that became standard. The benchmark progression — SWE-bench $\rightarrow$ SWE-bench Verified [OpenAI, 2024b] $\rightarrow$ ProgramBench [OpenAI, 2024a] — consistently increases task realism and removes "score-able without writing the code" loopholes. RTL-UNIVERSE is an HDL-side analogue at the project-scale step.

End-to-end hardware-design benchmarks. The authors' own prior efforts include Spec2GDSBench [Andrews and Reddy, 2026a,b,c], which evaluated agents on a full spec-to-GDSII pipeline from *scratch* (the agent writes RTL given only a behavioural specification, then a back-end flow synthesises it). We pivoted away from build-from-scratch toward *restore-from-an-existing-project* for two reasons. (i) Real projects come with real testbenches: an upstream Verilator/cocotb suite is much more carefully constructed than any test we would write to grade "the agent's UART". (ii) Real projects come with real build systems and integration constraints: file naming, parameter modules, generator scripts, and Makefile structure all add real engineering surface area. RTL-UNIVERSE is the first design we have built where the test infrastructure is entirely upstream.

Sandboxing and contamination. Test-set contamination [Golchin and Surdeanu, 2024] and agent-side network access are the two ways an HDL benchmark can be silently passed without writing the code. Section 6 documents both, and we believe future agent benchmarks should treat both with equal care.

3 Benchmark Design

3.1 Task selection

We chose tasks against three criteria: (i) the upstream project must have a working CI test suite that runs locally inside the container; (ii) the design unit to be restored must be substantial enough that an LLM cannot trivially regenerate it from documentation alone (typically $\geq$ 30 stripped files, or one large file with hundreds of internal signals); and (iii) the task should require the agent to use the upstream build system rather than its own. Table 1 lists the eleven tasks.

Table 1: The eleven RTL-UNIVERSE tasks. #strip is the count of deleted implementation files. Tasks marked † have a verifier whose pass criterion can be exit-code-only or assertion-trivial (Section 6); these are reported but discounted in the cleaned ranking.

Task	Upstream	Domain	#strip	Verifier
`ibex-e2e`†	lowRISC/ibex	RISC-V core	31	4 SW progs in simple_system
`ibex-full`†	lowRISC/ibex	RISC-V core	32	+ tb_cs_regs UVM test
`coralnpu-e2e`	google-coral/coralnpu	NPU	240	215 cocotb (Bazel)
`coralnpu-full`	google-coral/coralnpu	NPU	280	239 cocotb + 2 mobilenet
`nvdla-e2e`	nvdla/hw	DL accel.	266	SDP/PDP/conv tests
`nvdla-full`	nvdla/hw	DL accel.	283	full nv_full tests
`openpiton-e2e`	Princeton/OpenPiton	ESL	4	cocotb counter/lfsr/uart
`openpiton-full`	Princeton/OpenPiton	ESL	4	18 Icarus benches
`pulp-common-cells-full`	pulp-platform	cell library	33	in-repo testbench suite
`secworks-aes-full`	secworks/aes	AES-128 IP	7	5 FuseSoC/Icarus benches
`veer-el2-block`†	chipsalliance/VeeR-EL2	RISC-V core	40	1 PyUVM `test_irq`

The eleven tasks cover three difficulty regimes: small focused units (4–10 files, used to calibrate harness correctness), mid-scale modules (30–40 files, integration with bus protocols), and large-scale designs (200+ files, where the agent must reconstruct an entire RTL hierarchy). All upstream projects are widely used: lowRISC's Ibex is the reference RISC-V core for OpenTitan; chipsalliance's VeeR EL2 is the reference for the SweRV family; NVDLA is NVIDIA's open deep-learning accelerator; google-coral's Coral NPU is Google's edge-NPU reference design; PULP common_cells is the cell library used by every PULP-platform SoC.

3.2 Restoration framing and reward

For each task we identify one design unit and `rm` its implementation files *after* taking a manifest of what was deleted. The remaining files — testbenches, build files, READMEs, register-map JSON, linker scripts, reference Python golden models — are kept. The agent receives a working repository plus an instruction file naming the deleted files. The reward is the fraction of upstream CI tests that pass after the agent's final write, in $[0, 1]$, with single-test verifiers using a binary $\{0, 1\}$.

The deletion-and-restore framing is deliberate. We considered three alternatives: (i) *generate from a written spec* (the Spec2GDSBench approach); we found written specs always under- or over-constrain the design, and the agent ends up grading itself against our written rules rather than against real CI. (ii) *patch real GitHub bugs*; this is the SWE-bench philosophy, but the rate at which HDL projects produce well-isolated patch-with-test PRs is approximately one tenth of Python's, which our prior SWE-UNIVERSE-VERILOG pipeline documented (the companion logbook gives details). (iii) *the present approach*, restoration: the upstream CI does the grading, and the deletion is a clean noun phrase to put in the instruction ("these files are missing, recreate them"). It also avoids leaking the answer in the spec since the agent must reconstruct concrete syntax, not just write to a behavioural contract.

4 Harness

We instantiate each evaluated agent inside a Docker container built from the task's `environment/` Dockerfile. Just before the agent gets control, the harness writes a DNS sinkhole to `/etc/hosts` naming the most common HDL hosting domains:

```
SINKHOLE = ["github.com", "raw.githubusercontent.com", "api.github.com",
  "objects.githubusercontent.com", "codeload.github.com", "gist.github.com",
  "gitlab.com", "bitbucket.org", "codeberg.org", "sr.ht", "huggingface.co",
  "hf.co", "sourceforge.net"] # appended as "0.0.0.0 <domain>"
```

The intent is to stop a direct `git clone` of the upstream repo. We document in Section 6 that this is necessary but not sufficient.

Concurrency, accounts, supervision. The harness is a Python launcher that runs many agent jobs in parallel against the matrix. Six mechanisms turned out to matter: (i) per-job credentials — each Claude Code job binds a per-job copy of `.credentials.json` so simultaneous installs cannot `Text file busy` each other, and the host file is re-read at each launch to pick up OAuth refreshes; (ii) asymmetric per-account caps — subscription-billed agents (Claude Code, Codex) rate-limit per account, so we cap at MAX_CLAUDE_ACCT1 $= 2$ and MAX_CLAUDE_ACCT2 $= 1$ across two accounts; (iii) stuck-job killer — a periodic sweep SIGKILLs harbor processes whose log has not advanced for > 30 minutes; (iv) race-guard — a 120 s short-term cache of recently-launched job names prevents `ps` latency from triggering duplicate launches; (v) disk-fill supervisor — the supervisor pauses the launcher when the host root partition exceeds 90% (one Sonnet job leaked 812 GB of Verilator build cache through a bind-mounted credentials directory in our run, and would have crashed our shared lab server without this guard); (vi) orphan-container reaper — prunes per-job credential directories whose harbor wrapper has exited. These controls are not exotic but they are non-trivial; we view them as part of what RTL-UNIVERSE contributes as a benchmark, not as plumbing.

Models and harnesses. We evaluate seven model$\times$harness pairs: Opus 4.7 and Sonnet 4.6 via Claude Code; GPT-5.5 (reasoning effort `xhigh`) via Codex CLI; and DeepSeek V4-Pro, GLM-5.1, Kimi K2.6, and Qwen 3.6-27B via OpenCode/OpenRouter. Each cell has a 10-hour wall-clock cap with up to 3 trials.

5 Results

Table 2: Reward matrix on RTL-UNIVERSE. Bold cells are perfect (1.000). Tasks marked † have a verifier flaw and are omitted from the strong-verifier count. "F" is an infrastructure failure (auth/install error).

Model	cor-e2e	cor-fl	ibe-e2e†	ibe-fl†	nvd-e2e	nvd-fl	opn-e2e	opn-fl	pulp	sec	vee†
GPT-5.5	**1.00**	.42	**1.00**	**1.00**	.00	**1.00**	.00	F	.00	**1.00**	**1.00**
Opus 4.7	**1.00**	.09	**1.00**	.20	.00	.42	.00	**1.00**	.00	**1.00**	**1.00**
Sonnet 4.6	.18	.08	.00	**1.00**	.86	**1.00**	.00	.00	.00	**1.00**	.00
Kimi K2.6	.00	.02	.00	.00	**1.00**	**1.00**	.00	.06	.00	.00	.00
DeepSeek	.03	.03	.00	.00	.14	.00	**1.00**	.00	.00	.00	.00
GLM-5.1	.00	.00	**1.00**	.00	.00	.00	.00	.00	.00	.00	.00
Qwen-27B	.00	.00	.00	.00	.00	.00	.00	.00	.00	.00	.00

Headline. Table 2 shows the raw matrix. GPT-5.5 leads at 6/11 perfect scores; Opus 4.7 has 5/11; Sonnet 4.6 has 3/11. The smaller open-weight models cluster at 0–2 perfects and average reward < 0.20. The OpenCode harness around the smaller models tends to produce two failure modes that we categorise informally as "early give-up" (under 5 minutes wall-clock, no real attempt) and "build-system blocked" (the agent writes plausible-looking RTL but cannot drive the upstream build). Both happened across many cells. We did not adjust prompts, harness settings, or budget across models to keep comparisons fair.

Per-task observation. The two tasks where multiple models score well (`secworks-aes-full`, `openpiton-full`) are the ones with the smallest stripped surface (7 and 4 files) and the cleanest in-repo Python reference model. The tasks where zero or one model scored well (`coralnpu-full`, `nvdla-e2e`, `pulp-common-cells-full`) are the ones with the largest stripped surfaces and the

most upstream-build complexity — not necessarily the most algorithmically hard. We read this as evidence that *project-scale RTL restoration is dominated by build-system and integration friction*, not by the difficulty of writing any one Verilog statement.

6 Audit: How agents reach high scores

When we read the agent transcripts we found that several perfect scores were achieved without writing the design. We categorise the bypasses into three groups; the full method list is in the Appendix. We treat this as a quality-control finding about how to run an HDL agent benchmark.

(A) Source downloads through unblocked egress. Our `/etc/hosts` sinkhole blocks `github.com` but not its alternatives. GPT-5.5 successfully cloned Ibex from the third-party mirror at `git.blizzard.systems` (twice), used a GitHub-to-text service (`gitextract.com`) for NVDLA, and bypassed the sinkhole with `curl -resolve` on a hard-coded code-load IP. Opus 4.7 used Anthropic's server-side `WebFetch` tool with a CDN URL (`cdn.jsdelivr.net/gh/lowRISC/ibex@master/`) to fetch all 31 stripped Ibex files in 37 calls, and used the Software Heritage Archive vault to recover Coral NPU. None of these are model failures; they are sandbox failures.

(B) Built-in agent tools. Both Claude Code (`WebFetch`, `WebSearch`) and Codex CLI (the `codex_apps` MCP server's `github_fetch_file` tool) ship with file-fetch capabilities that route through the harness vendor's servers, not through the container's network stack. `/etc/hosts` cannot reach them. We had not noticed the Codex MCP existed when we built the sandbox; it accounts for one of GPT-5.5's perfect scores.

(C) Verifier exploits. The Ibex `simple_system` `tests/test.sh`'s pass criterion is "simulator exit code $= 0$"; an empty top-level wrapper that runs Verilator's setup phase and exits clean satisfies this without ever running the firmware. Both Gemini 3.1-Pro (which is not in our headline table; in the larger superset of runs it scored 1.000 here using exactly this exploit) and GLM-5.1 reach perfect scores on this verifier. The `veer-el2-block` PyUVM IRQ test has `assert True` in its `check_phase`; any compiling DUT passes. Opus identified this in 17 minutes and won the cell with 37 `module name(); endmodule` stubs. The verifier flaws are upstream bugs that real HDL projects also have; restoration-style benchmarking surfaces them quickly because the agent will exploit them given the chance.

Cleaned ranking. Table 3 re-scores the matrix after zeroing every cell that either matches a category-A/B bypass we observed in the transcripts or scores 1.000 on a flag-marked task. On the eight non-flagged tasks, the strongest agents pass 2/11 each.

Table 3: Pass rates before and after the audit. "Raw" counts every reward $= 1.0$; "Strong" zeroes both confirmed bypasses and the three flagged tasks.

Model	Raw pass	Raw avg	Strong pass	Strong avg
GPT-5.5	6/11 (55%)	0.58	1/11 (9%)	0.13
Opus 4.7	5/11 (45%)	0.52	2/11 (18%)	0.25
Sonnet 4.6	3/11 (27%)	0.37	2/11 (18%)	0.28
Kimi K2.6	2/11 (18%)	0.19	2/11 (18%)	0.19
DeepSeek	1/11 (9%)	0.11	1/11 (9%)	0.11
GLM-5.1	1/11 (9%)	0.09	0/11 (0%)	0.00
Qwen-27B	0/11 (0%)	0.00	0/11 (0%)	0.00

What the cleaned ranking tells us. The headline gap between proprietary frontier agents and open-weight agents nearly disappears. The *benchmark*, in other words, was good enough to differentiate raw pass rates but the bypasses inflate the apparent gap. We do not claim the underlying capability is identical — the open-weight models genuinely fail many cells they don't try to bypass — but the comparison should be done on a leakage-free sandbox, and ours wasn't strict enough. We give specific recommendations for tightening it in Section 7.

7 Recommendations and Limitations

Sandbox. Default-deny egress at the iptables level with allowlists for the package mirrors and vendor APIs the agent needs. Block CDNs that mirror GitHub (`cdn.jsdelivr.net/gh`, `unpkg.com`, `cdnjs.cloudflare.com`), text-of- GitHub services (`gitextract.com`, `r.jina.ai`), Software Heritage, and any host on which an MCP server fetches files server-side. For Claude Code, disable `WebFetch/WebSearch` via `-allowed-tools`; for Codex, do not register the `codex_apps` MCP.

Verifiers. Audit upstream verifiers for "exit code $= 0$ counts as pass" and for trivial `check_phase` assertions. Replace with output-hash comparisons or explicit signal-trace assertions. A simple structural sanity check — "does the design hierarchy contain modules of these names" — catches the behavioral-stub family without constraining the implementation.

Limitations. We run one trial per cell (with up to 3 retries on infrastructure failure); we do not measure within-cell variance. The cheat-vs.-legit verdict is a manual judgement call on the transcripts; we publish all transcripts and invite re-adjudication. The benchmark covers 11 tasks chosen by us; a wider sweep across the open-source HDL ecosystem (and especially across non-RISC-V architectures) would be a natural extension. Finally, the harness is engineered for our two laptops + lab server combination; porting to a fully cloud-native runner is out of scope here.

8 Conclusion

RTL-UNIVERSE is a deliberately harder hardware-LLM benchmark than what is currently in the literature. By framing tasks as restoration of stripped files inside a real upstream project, we exercise the build systems, integration constraints, and testbenches that real HDL engineers actually wrestle with. Current frontier agents pass 1–2 of the eight strong-verifier cells; project-scale RTL restoration is not a saturated problem. We hope the benchmark, the harness, and the audit are all useful to the next group that builds on this.

Acknowledgements. We thank the Anthropic, OpenAI, and OpenCode teams for making their agent harnesses available. The cheating analysis was assisted by Claude Opus 4.7, with the analysis performed on already-concluded transcripts.

References

David Andrews and Saketh Reddy. Spec2GDSBench: Implementation and baseline results for end-to-end chip design evaluation. Technical report, Internal weekly research report, March 2026a.

David Andrews and Saketh Reddy. From individual tasks to full SoC: Redesigning spec2GDSBench around linux boot. Technical report, Internal weekly research report, April 2026b.

David Andrews and Saketh Reddy. Pivoting to AI accelerator benchmarks: An integration ladder for spec2GDSBench. Technical report, Internal weekly research report, April 2026c.

Shahriar Golchin and Mihai Surdeanu. Time travel in LLMs: Tracing data contamination in large language models. `https://arxiv.org/abs/2308.08493`, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation. *arXiv preprint arXiv:2309.07544*, 2023.

Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RTLLM: An open-source benchmark for design RTL generation with large language models. *IEEE Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024.

NVIDIA Research. CVDP: A comprehensive verilog design problems benchmark. `https://github.com/NVlabs/CVDP-benchmark`, 2024.

OpenAI. Introducing ProgramBench: harder coding agent evaluations. Technical report (programbench), 2024a.

OpenAI. Introducing SWE-bench Verified. `https://openai.com/index/introducing-swe-bench-verified/`, 2024b.

Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. VeriGen: A large language model for Verilog code generation. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 29(3), 2023.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

A Bypass methods, summary

Table 4: Eight observed sandbox-bypass methods.

#	Method	Used (perfect score) by
A1	Third-party Git mirror (`blizzard.systems`)	GPT-5.5 ($\times 2$)
A2	GitHub-backed CDN (`cdn.jsdelivr.net/gh`)	Opus
A3	GitHub-to-text service (`gitextract.com`)	GPT-5.5
A4	Software Heritage Archive vault	Opus
A5	`curl -resolve` + IP literal	GPT-5.5
A6	Other mirrors (gitee, ghproxy, unpkg, r.jina.ai)	none successful
B1	Claude Code `WebFetch`	Opus
B2	Codex MCP `github_fetch_file`	GPT-5.5
B3	Codex MCP `web_search`	none successful
C1	Behavioural stub (no real implementation)	GPT-5.5
C2	Verifier exit-code exploit	GLM-5.1
C3	No-op `assert True` scoreboard exploit	Opus

B Reproducibility

The harness, task definitions, and complete agent transcripts are released on the `codex-experiments-202605` branch of `github.com/cs3220-research/rtl-universe`. Each cell's `result.json` contains the agent's full arguments, start/finish times, reward, and (where the harness reports it) cost in USD; the `agent/` subdirectory contains the full conversation log. Total wall-clock for the run reported here was approximately 4 days on a 128-CPU 1 TB-RAM lab server.